Assignment-9

* Problem Statement:

Queues are frequently used in computer programming & a typical example is the creation of a job queue by operating system, if the operating system does not use priorities then jobs are processed in order that enter system.

* Objectives:

i) Understand the concept of queue & FIFO
ii) Implement a linear queue using array
iii) Implement function to add & delete jobs.
iv) Simulate processing of jobs in order they enter the system.
v) Analyze the time & space complexity of queue.

* Psuedo Code:

```
Start
Initialize front = -1 , rear = -1
Repeat
  Display menu
  Accept choic
If choice = 1
      If rear = Max-1
        Display = "Overflow"
    Else
      accept job
      If front = -1,
          front++,
          rear ++,
```

```
    ENDIf
Else If choice=2
    If front=-1
        Display "Underflow"
    Else
        process & delete job from front
        front++,
    End If

Else If choice = 3
    If front=-1
        Display "Queue Empty"
    Else
        print from front
    End If

Else
    EXIT

STOP.
```

* Timpe Complexity

Operation	Complexity
Enqueue	O(1)
Dequeue	O(1)
Display	O(n)

* Conclusion: The job queue simulation demonstrate the FIFO Principle of linear queue, jobs are added at rear & processed /deleted from the front in same order in which they enter the system.

* FAQs:

Q1 What is Queue?

A- A queue is linear data structure in which elements are interested from the rear & deleted from front.

Q2. What is FIFO?

A- FIFO means, First In Fist Out.

- The elements inserted first will be the one that are deleted first.

Q3. Why is queue suitable for an operating System job queue?

A- It is suitable because it processes jobs in the same order in which they are entered, ensuring fairness to them.

Q4. What is Queue Overflow & Queue Underflow?

A- Overflow:

- When we try to insert an element into a queue but the queue is full it is overflow.

- Under flow:

When we try to delete an element from a queue but the queue has no element then it is underflow.

Q5 What are application of Queue?

A- i. CPU Scheduling
ii. Printer Scheduling
iii. Job Scheduling
iv. Network data Handling
v. Customer Service Systems.

Q6. What is Circular queue?

A- A circular queue in which the last position is connected to the first position of the same queue.

Q7. Why is Circular queue preferred over a linear queue?

A- Circular queue reuses the empty spaces created after deletion making better use of memory & better code optimization.

Q8. What principle does circular queue follow.

A- Circular Queue follows the First In First Out [FIFO] principle, same as linear queue.

Q9. What are the basic operations performed on circular queue?

A- Enqueue - Insertion of data

- Dequeue - Deletion of data

- Display - Printing data.

Q0. What is an overflow & Underflow condition in circular queue?

A- Queue Overflow: If the value of rear = MAX, then queue is overflow, means it is full.

- Queue Underflow: If the value of front = -1, then queue is underflow, means it is empty

Experiment 09: Job Queue Simulation Using Linear Queue

C Source Code

```
#include <stdio.h>
#include <string.h>

#define MAX 5
#define NAME_LEN 50

typedef struct {
    int id;
    char name[NAME_LEN];
} Job;

Job queue[MAX];
int front = -1, rear = -1;

void addJob(void) {
    Job job;
    if (rear == MAX - 1) {
        printf("\nQueue Overflow! Cannot add more jobs.\n");
        return;
    }

    printf("Enter Job ID: ");
    if (scanf("%d", &job.id) != 1) {
        printf("Invalid Job ID.\n");
        while (getchar() != '\n') {}
        return;
    }
    printf("Enter Job Name (one word): ");
    scanf("%49s", job.name);

    if (front == -1)
        front = 0;
    queue[++rear] = job;
    printf("Job added successfully.\n");
}

void deleteJob(void) {
    if (front == -1 || front > rear) {
        printf("\nQueue Underflow! No jobs to process.\n");
        return;
    }

    printf("\nProcessing Job: ID=%d, Name=%s\n",
           queue[front].id, queue[front].name);
    front++;

    if (front > rear)
        front = rear = -1;
}

void displayJobs(void) {
    int i;
    if (front == -1) {
        printf("\nNo pending jobs in the queue.\n");
        return;
    }

    printf("\nPending jobs (front to rear):\n");
    for (i = front; i <= rear; i++)
        printf("ID=%d, Name=%s\n", queue[i].id, queue[i].name);
}

int main(void) {
    int choice;
    do {
        printf("\n--- JOB QUEUE SIMULATION ---\n");
        printf("1. Add Job\n2. Process/Delete Job\n3. Display Jobs\n4. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1) {
            printf("Invalid choice.\n");
            return 1;
        }

        switch (choice) {
            case 1: addJob(); break;
            case 2: deleteJob(); break;
            case 3: displayJobs(); break;
            case 4: printf("Exiting program.\n"); break;
            default: printf("Invalid choice. Try again.\n");
        }
    } while (choice != 4);

    return 0;
}
```

Sample Input / Output Screenshot

Terminal-style screenshot generated from a compiled test run of the program.

